Smart & Green Gaming: Using GenAI to Automate Mobile Game Energy Testing

Cristiano Zhou
University of Lisbon, LASIGE
Lisbon, Portugal
czhou@lasige.di.fc.ul.pt

Wellington Oliveira
University of Lisbon, LASIGE
Lisbon, Portugal
woliveira@fc.ul.pt

Abstract

Mobile games are among the most demanding applications on smartphones, yet their energy consumption is hard to measure rigorously, since manual play cannot be repeated exactly and fixed automation scripts break on the unpredictable elements of games. This work investigates whether generative AI can help build reproducible energy tests for mobile games. From screenshots, an LLM iteratively generated Python scripts that control each game; these were converted into Java UI Automator tests and executed through the ebserver framework on a physical Android device. The approach was applied to four games of increasing complexity (Flow Free, Summoners War: Sky Arena, Raid: Shadow Legends and Pokémon TCG Pocket) across 20 configurations with 30 runs each, producing tests that run without human intervention. The results show that the effect of in-game settings depends on the game: animations and a reduce mode had no measurable effect in lighter games, whereas lowering the frame rate reduced consumption by 6–27% in the more demanding ones, and faster gameplay saved 7.5% simply by shortening the session. Not all results matched expectations, and the analysis revealed practical challenges, such as variations in screen brightness that had to be corrected for.

Keywords

Mobile Energy Consumption, Green Software, Android Automation, UI Automator, Generative AI, Energy Benchmarking

1 Introduction

Smartphones rely on a limited battery, and the energy consumption of applications has a direct impact on user experience. Mobile games are among the most widely used applications and, in general, among the most demanding on hardware, which makes their energy consumption a relevant concern, both for players and from the perspective of sustainable software.

Measuring this consumption rigorously is, however, difficult. Energy tests of mobile games often rely on people playing the game, which takes a long time and lacks rigour: different people play in different ways, and even the same person does not repeat exactly the same actions from one session to the next. Traditional automation tools, on the other hand, are based on fixed sequences of actions, which easily fail when faced with the unpredictable elements of games, such as pop-ups, events or battles of variable duration.

The goal of this work is to make these tests automatic with the support of generative AI. From screenshots, an LLM generates the code that controls the game; the resulting test plays the game on its own and is executed repeatedly through the ebserver framework, making it possible to change the game's settings and compare consumption under controlled conditions. This approach was applied to four games (Flow Free, Summoners War: Sky Arena, Raid: Shadow Legends and Pokémon TCG Pocket) for a total of 20 configurations with 30 runs each. The results show that the effect of settings depends on the type of game, and that usage time can matter more than graphics quality.

The remainder of this report is organised as follows: Section 2 presents the background, Section 3 the motivation, and Section 4 the research questions; Sections 5 and 6 describe the methodology and the experimental setup; Section 7 presents the results, Section 8 discusses them, and Section 9 concludes.

2 Background and Context

Energy efficiency is no longer a concern only of hardware designers: the way application software is written has a direct impact on how much energy it consumes, yet developers often lack the knowledge and tools to measure and reduce it [15]. On mobile devices this concern is particularly visible, since energy consumption translates directly into battery life.

Android does not measure the consumption of each component directly. Instead, each manufacturer provides a power profile that specifies the current drawn by each hardware component in each of its states, for example the screen at minimum and maximum brightness or the CPU at each frequency, and the framework multiplies these values by the time each component spends in each state to estimate the charge consumed, in mAh [2]. These estimates are exposed through `dumpsys batterystats`, which ebserver [13] uses to collect data. A closely related study uses the same framework to quantify the energy impact of user-controllable settings such as brightness, refresh rate and battery-saving profiles across popular apps [12], the scope is similar to what is done here, but applied to messaging, social media and video apps instead of games. Two consequences of this model are relevant to this work: a component for which the manufacturer defines no coefficient is not billed at all, and the screen is billed according to the brightness level at which it was kept.

Reproducible energy tests require the same interaction to be executed repeatedly. For this reason, UI automation frameworks, which mimic user interaction while energy is profiled, are considered the most reliable approach for energy testing on mobile devices [5]. The choice of framework matters, since the framework itself adds an energy overhead, and UI Automator is among those recommended as a good alternative [5]. UI Automator supports black-box tests that interact with any application visible on the device and run as standard JUnit tests [1]. Games often draw their interface through a game engine on a single surface, so their buttons are frequently not exposed as individual UI elements. Tests must then rely on screen

coordinates and fixed sequences of actions, which break when the game shows an unexpected pop-up, an event or a longer battle. Framework choice adds its own variability: a comparison of four web UI automation frameworks found that the energy cost of the same action can vary by a factor of six depending on the framework used [7].

Beyond the choice of framework, prior work has catalogued concrete energy-efficiency practices for mobile applications, based on an analysis of commits, issues and pull requests from over 1 700 open-source Android and iOS apps [4], and identified categories of Android API calls and usage patterns with disproportionately high energy consumption, measured with a hardware power monitor across 55 apps [8]. A crowdsourced dataset of over 23 million battery samples from more than 1 600 device brands offers a complementary, large-scale perspective, showing how much variability exists across real-world usage compared with the controlled, single-device setting adopted here [14]. At a finer grain, Chen et al. measure the energy cost of Web Push notifications in the Telegram mobile web app, isolating a single API in a controlled setting [3]; that approach is similar in spirit to the per-setting comparisons carried out here, though applied to an API rather than to a game's own configuration options.

Recent work has started to use large language models to drive mobile GUI testing. GPTDroid, for instance, passes information about the current screen to an LLM, which generates test scripts that are executed and whose results are fed back to the model in an iterative loop [10], and vision-driven approaches based on multimodal LLMs work directly from screenshots [11]. These approaches focus on exploring applications and finding bugs, rather than on producing identical, repeatable runs for energy measurement. The work presented here applies a similar iterative, screenshot-based use of an LLM, but for a different goal: building reproducible energy tests for mobile games.

The screen is consistently the dominant source of consumption in this study (Section 8). This matches prior findings on OLED displays: switching to dark mode saves only 3–9% of power at typical indoor brightness levels, but 39–47% at maximum brightness [6]. The choice of colour palette itself can also be optimised for energy, contrast and visual consistency at the same time [9]. This sensitivity to brightness matches the variation observed in the data collected for this work (Section 6).

3 Motivation and Project Framing

Although energy consumption is an important non-functional requirement of mobile applications, it is still often tested in an ad hoc way [5]. For games, this problem is amplified: the same session can unfold differently every time, which makes manual measurements hard to repeat and hard to compare. The motivation for this project was therefore to change the way these tests are carried out, replacing manual play with automated tests that can be repeated many times under the same conditions, so that differences in consumption can be attributed to the settings being changed rather than to the way the game was played.

The project is framed as a controlled comparison: the independent variables are the settings offered by each game, and the dependent variable is energy consumption, measured over repeated runs of the same automated task. The four games were not chosen at random, but to cover increasing levels of complexity. Flow Free is a simple 2D puzzle game, whose only relevant setting is its animations. Summoners War: Sky Arena is a simple 3D game, offering combat speed and a reduce mode. Raid: Shadow Legends is a more complex 3D game with several graphics options, of which quality and frame rate were tested. Pokémon TCG Pocket is a card game with many 3D animations and the same two types of setting. This progression makes it possible to observe whether the effect of in-game settings changes as games become more demanding, and to test the automation approach on games of very different natures.

4 Research Goals

The main goal of this work is to assess whether generative AI can be used to automate energy consumption tests for mobile games in a reproducible way, and to use those tests to measure the impact of the games' own settings. This goal translates into three research questions:

RQ1: Can an LLM, working from screenshots, generate scripts that automate games with non-deterministic interfaces and produce reproducible energy tests?

RQ2: Do the settings available in the games (graphics quality, frame rate, animations and combat speed) produce measurable differences in energy consumption?

RQ3: What are the main limitations of this approach, and what would be needed to overcome them?

The initial hypothesis is that settings with a heavier graphics load increase consumption, and that this effect is more visible in more demanding games.

5 Methodology

5.1 LLM-Assisted Script Generation

The automation process relies on an iterative human–LLM approach (Figure 1): a screenshot of the game is captured, the LLM identifies the current screen state and generates the ADB command for the next step as a Python script, and the cycle repeats until each game's stopping point is reached (10 levels in Flow Free, one season in Raid and Sky Arena, and 3 battles in Pokémon TCG Pocket), defined by the game's natural cycle.

The Python script, validated on an emulator, is then converted into a JUnit UI Automator test in Java and integrated into ebserver, the energy measurement framework the mentor provided. Python enables rapid iteration, while Java ensures compatibility with ebserver and reproducible data collection.

5.2 Coordinate Handling

Coordinates are stored as fractions of the framebuffer, making the scripts portable across resolutions. Taps are executed with `input touchscreen` through `executeShellCommand`, avoiding the mismatch between the display and framebuffer coordinate systems.

5.3 Screen Stability Detection

To handle unpredictable changes on screen, a stability detection mechanism compares consecutive frames using the mean absolute

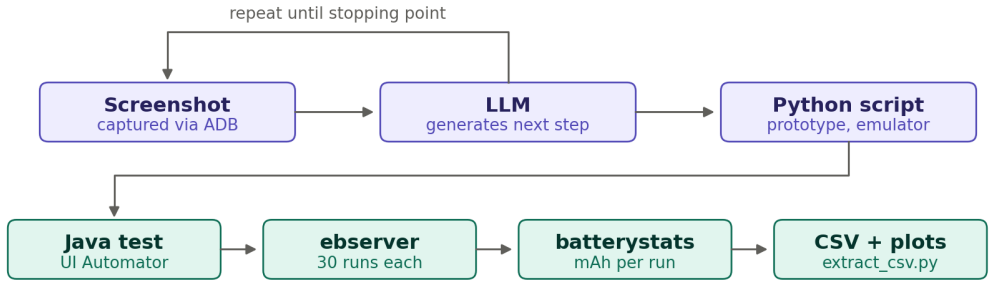


Figure 1: Overview of the automated energy testing pipeline. In the first phase, screenshots are iteratively sent to an LLM that generates the ADB command for the next in-game action, producing a Python prototype script validated on an emulator. In the second phase, the validated flow is reimplemented as a Java UI Automator test and executed on a physical device through the ebserver framework, which collects battery statistics and consolidates them into CSV files and comparative plots.

Table 1: Experimental configurations evaluated per game.

Game	Factors varied	Configurations
Flow Free	Animations on / off	2
Sky Arena	Speed x1/x2/x3 × normal / reduce	6
Raid	Quality (M/H/U) × 30/60 fps	6
Pokémon	Quality (L/M/H) × 30/60 fps	6
Total		**20**

difference (MAD) of 32 × 32 grayscale thumbnails, replacing fixed waiting times.

Raid: Shadow Legends. The mechanism makes it possible to close a variable number of pop-ups without hard-coding their number.

Pokémon TCG Pocket. The end-of-battle screen contains constantly moving elements and never stabilises, so a two-level check was designed instead. The script taps every 30 seconds and checks stability again. If the screen keeps changing, the battle is still in progress. If it becomes static, the first post-battle pop-up has been reached.

6 Experimental Setup

Formal data collection took place on a physical device, because the BatteryManager API returns fixed values on an emulator. The emulator was used only to validate coordinates and prototype flows, which is why the architecture in Figure 1 has two phases.

All runs were executed on a Samsung SM-G780G with maximum brightness, the dark theme and the system battery saver disabled. Before each run, the `experiments_queue.json` file was deleted and battery statistics were reset with `adb shell dumpsys batterystats --reset`. In Flow Free, WiFi was disabled to avoid advertisements.

Twenty configurations were evaluated (Table 1), with 30 runs each. Data were extracted with ebserver's `extract_csv.py` script, and runs flagged as outliers were excluded.

Per-run analysis showed, however, that brightness did not always remain at its maximum. The screen consumption per second takes discrete levels: 0.093 mAh/s at maximum brightness, and 0.078 or 0.062 mAh/s at lower levels. This affected 7 Sky Arena runs (x3, reduce), all already flagged as outliers, 25 Raid runs (60 fps, medium) and most Pokémon configurations. The cause could not be determined; automatic dimming after inactivity is a plausible explanation, since the Pokémon script waits up to 30 seconds between taps. For this reason, comparisons in Raid and Pokémon use the *consumption outside the screen* per second (total minus screen), which does not depend on brightness and corresponds to the part of consumption that in-game settings can affect. This diagnostic step is reported in mAh/s, the unit in which `batterystats` records charge; all consumption figures reported in Section 7 onward, however, are converted to energy in joules ($E = \frac{\text{mAh}}{1000} \times V \times 3600$, using the battery voltage V recorded for each run), since mAh measures charge rather than energy and is therefore not directly comparable across runs at different voltages.

7 Results

7.1 Flow Free

With animations disabled, total consumption was slightly higher (860.9 vs 840.8 J, +2.4%), but because each run took longer (419.3 vs 408.5 s). Normalised by duration, disabling animations reduces the energy rate by only 0.25% (2.053 vs 2.058 J/s). CPU consumption is indeed lower without animations (124.1 vs 127.5 J), but the screen accounts for about 70% of the total, so this saving is diluted. In a visually simple game such as Flow Free, disabling animations has no practical effect.

7.2 Summoners War: Sky Arena

Combat speed has a clear effect: consumption drops from 1272.6 J (x1) to 1215.1 J (x2) and 1177.6 J (x3), a reduction of 7.5% ($p < 10^{-40}$; Table 2, Figure 2). The energy rate, however, is practically the same at all three speeds (2.17–2.20 J/s): the saving comes solely from a shorter session.

Reduce mode produced no measurable saving in total energy. At x1 and x3, the differences from normal mode (+0.25% and +0.05%)

Table 2: Energy consumption of Summoners War: Sky Arena across the six evaluated configurations. n is the number of valid runs after excluding outliers, out of 30 runs per configuration.

Mode	Speed	n	J	Duration (s)	J/s
Normal	x1	29	1272.6	587.7	2.165
Normal	x2	29	1215.1	555.3	2.188
Normal	x3	29	1177.6	535.1	2.201
Reduce	x1	29	1275.8	597.1	2.137
Reduce	x2	28	1190.3	555.2	2.144
Reduce	x3	22	1178.1	534.9	2.202

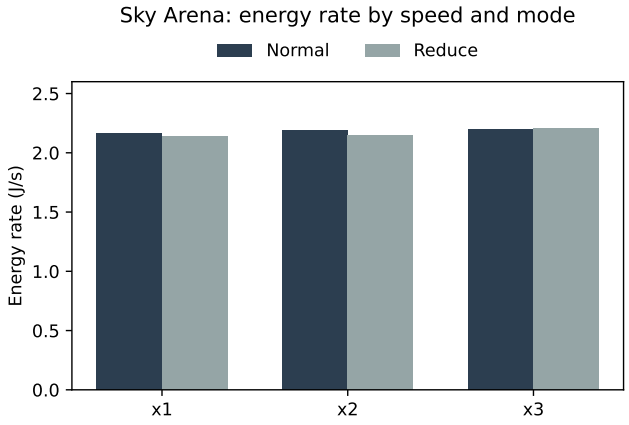


Figure 2: Energy rate (J/s) of Summoners War: Sky Arena, by combat speed and mode. The rate is practically unchanged across speeds and modes; the energy savings in Table 2 come from shorter session duration, not from a lower rate.

are of negligible magnitude ($p = 0.038$ and $p = 0.81$); at x2, reduce mode consumes 2.04% *less* ($p < 10^{-19}$), the only case with a direction favouring reduce mode, but still too small to be of practical relevance. As this setting is presented to the player as a way of reducing resource usage, its lack of effect is worth noting, although it is limited to the conditions tested.

Of the 8 outliers in the x3 reduce-mode configuration, 7 correspond to runs in which brightness dropped (Section 6), so their exclusion does not indicate a problem with the configuration itself.

7.3 Raid: Shadow Legends

Using the consumption outside the screen (Table 3, Figure 3), the effect of quality at 30 fps is monotonic: +4.2% from medium to high ($p = 0.028$) and +11.4% from high to ultra ($p < 10^{-4}$). At 60 fps, moving from medium to high increases consumption by 14.3% ($p < 10^{-11}$), but ultra consumes 4.3% less than high ($p = 0.021$), an inversion the data do not explain. Lowering the frame rate from 60 to 30 fps reduces consumption at every quality level: by 11.3%, 19.2% and 6.0%. Raid is thus the first game in this study in which graphics settings have a clear effect.

Table 3: Consumption outside the screen (J/s) for Raid: Shadow Legends and Pokémon TCG Pocket. Quality levels are Medium, High and Ultra for Raid, and Low, Medium and High for Pokémon.

Game	Frame rate	Lowest	Middle	Highest
Raid	60 fps	0.769	0.879	0.841
Raid	30 fps	0.682	0.710	0.791
Pokémon	60 fps	0.934	0.949	1.065
Pokémon	30 fps	0.843	0.778	0.776

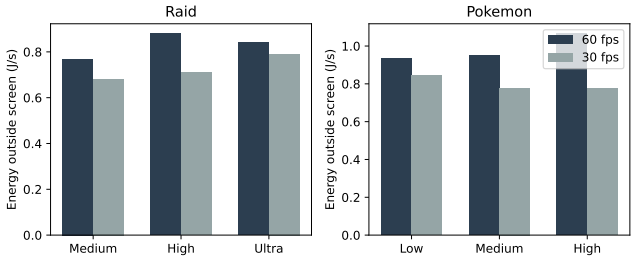


Figure 3: Energy rate outside the screen (J/s) for Raid: Shadow Legends and Pokémon TCG Pocket, by graphics quality and frame rate. Lowering the frame rate from 60 to 30 fps reduces the rate at every quality level in both games.

7.4 Pokémon TCG Pocket

Pokémon was the game most affected by the brightness variation, so total consumption is not comparable across configurations and the analysis uses only the consumption outside the screen (Table 3, Figure 3). Before this correction, the data suggested that at 30 fps high quality consumed less than medium; with the correction, this effect disappears.

Frame rate has the clearest effect: lowering it from 60 to 30 fps reduces consumption at every quality level, by 9.8%, 18.0% and 27.1% ($p < 10^{-8}$ in all cases). Graphics quality matters only at 60 fps: low and medium are indistinguishable (+1.6%, $p = 0.11$), and high increases consumption by 12.2% ($p < 10^{-9}$). At 30 fps, low and medium differ by −7.6% ($p < 0.001$) while medium and high are indistinguishable ($p = 0.91$), which suggests that, once the frame rate is capped, higher quality no longer carries a consistent, measurable cost.

8 Discussion

Whether an in-game setting changes energy consumption depends strongly on the game. Flow Free's animations and Sky Arena's reduce mode had a negligible effect. Raid and Pokémon were different: raising graphics quality increased the consumption outside the screen by 4–14% in Raid and up to 12% in Pokémon, and lowering the frame rate reduced it at every quality level tested (6–19% in Raid, 10–27% in Pokémon).

One reason for this may be the size of the consumption outside the screen, since that is the only part a setting can change. It is around 0.61 J/s in Flow Free and 0.74 in Sky Arena, but reaches 0.78

in Raid and 0.89 in Pokémon, against 1.448 J/s for the screen alone at maximum brightness. When the screen dominates, there is little left to reduce; in the heavier games the margin is bigger. So the saving a setting can realistically deliver seems to depend on how much computation the game does in the first place.

Usage time turned out to matter more than graphics settings in at least one case. Raising Sky Arena's speed from x1 to x3 saved 7.5%, the largest saving in this study, without changing the energy rate at all: the game simply finishes sooner. Reduce mode, which is marketed to the player as a way to save resources, had no measurable effect. Checking that kind of claim is exactly what an automated, repeatable test is good for.

These numbers come from a single device, and there is no way to tell from this data whether they would hold on other phones. Variability also differs a lot between games: the coefficient of variation of run duration was 0.07–0.14% in Sky Arena, 1.2–2.0% in Raid, and 1.5–3.5% in Pokémon, where every battle has different opponents and cards. So reproducibility does not just depend on the automation, it depends on how deterministic the game itself is. The brightness issue is a reminder of the same point: a condition assumed to be fixed still needs to be checked in the data, not just in the setup.

This also points to why generative AI matters for this kind of work. The LLM here was used to build the tests, which is what made it possible to automate four very different games without writing everything from scratch. Once built, though, each test runs a fixed sequence, and every unexpected situation needed its own hand-written fix: stability detection for Raid's pop-ups, a 30-second tap loop for Pokémon, disabling WiFi for Flow Free. None of these fixes generalise. Anything the script did not anticipate, a new in-game event, a window that pops up out of nowhere, a battle that just runs long, can send the test off its expected path.

The natural next step is to move generative AI into the execution itself, so the test can look at the screen and decide what to do next, the way a player would, instead of anticipating every case in advance. That comes with its own problems, though. Running inference costs energy and could interfere with the very thing being measured, and if decisions vary from run to run, reproducibility suffers. Working out how much flexibility to give the AI without losing the rigour the measurement needs is the main open question for continuing this work.

9 Conclusions and Future Work

To answer RQ1: yes, an LLM can be used to build energy tests for mobile games. From screenshots, it generated Python scripts that were converted into JUnit UI Automator tests and run 30 times per configuration through ebserver, across four quite different games. Once a test is built, it needs no human intervention to run; you start it and the game plays itself. Building it in the first place is another matter, and that step still needs a person, mostly to work through whatever is specific to each game.

RQ2 does not have a single answer, since it depends on the game. Flow Free and Sky Arena's reduce mode showed no relevant differences. Raid and Pokémon did, mostly through frame rate: dropping from 60 to 30 fps cut consumption by 6–27% depending on quality level. So the original hypothesis holds, but only for the more demanding games, not across the board. Session length mattered too, arguably more: in Sky Arena, playing at x3 instead of x1 saved 7.5% without the energy rate changing at all.

For RQ3, the biggest limitation comes from the games themselves rather than from the automation. A test always runs the same sequence of actions, but the games do not always respond the same way; things like the opponent faced, the cards drawn or how long a battle runs vary on their own. That variation was reduced, not removed. On top of that, every unexpected situation needed its own manual fix, and every number reported here comes from a single device.

The obvious next step is to bring generative AI into execution as well, so a test can react instead of just following a script. That has a real cost: less control over what happens at each moment of the game. We think the trade-off is worth it, but that is still a hypothesis, and testing it means measuring how much energy the AI itself uses and whether its decisions stay reproducible across runs. Trying the same experiments on other devices is the other obvious extension.

Acknowledgments

This work was carried out at LASIGE, University of Lisbon, within the Summer of Research programme.

References

[1] Android Developers. 2026. Write Automated Tests with UI Automator. https://developer.android.com/training/testing/ui-automator. Accessed September 2026.

[2] Android Open Source Project. 2026. Power Profiles for Android. https://source.android.com/docs/core/power. Accessed September 2026.

[3] Yu Chen, Jinhai Liao, Shutong Cai, Yiming Zhao, Yichen Tong, Ishas Kulkarni, and Ivano Malavolta. 2024. An Empirical Evaluation of the Energy Consumption of using Web Push APIs in Mobile Web Apps – The Case of Telegram. In *Proceedings of the IEEE/ACM 11th International Conference on Mobile Software Engineering and Systems (MOBILESoft)*. 22–32. doi:10.1145/3647632.3647987

[4] Luis Cruz and Rui Abreu. 2019. Catalog of energy patterns for mobile applications. *Empirical software engineering* 24, 4 (2019), 2209–2235.

[5] Luís Cruz and Rui Abreu. 2021. On the Energy Footprint of Mobile Testing Frameworks. *IEEE Transactions on Software Engineering* 47, 10 (2021), 2260–2271. doi:10.1109/TSE.2019.2946163

[6] Pranab Dash and Y. Charlie Hu. 2021. How Much Battery Does Dark Mode Save? An Accurate OLED Display Power Profiler for Modern Smartphones. In *Proceedings of the 19th Annual International Conference on Mobile Systems, Applications, and Services (MobiSys)*. ACM. doi:10.1145/3458864.3467682

[7] Timmie MR Lagermann, Kristina Sophia Carter, Su Mei Gwen Ho, Luís Cruz, Kerstin Eder, and Maja H Kirkeby. 2026. Framework Matters: Energy Efficiency of UI Automation Testing Frameworks. In *Proceedings of the 41st ACM/SIGAPP Symposium on Applied Computing*. 1636–1645.

[8] Mario Linares-Vásquez, Gabriele Bavota, Carlos Bernal-Cárdenas, Rocco Oliveto, Massimiliano Di Penta, and Denys Poshyvanyk. 2014. Mining Energy-Greedy API Usage Patterns in Android Apps: An Empirical Study. In *Proceedings of the 11th Working Conference on Mining Software Repositories (MSR)*. ACM, 2–11. doi:10.1145/2597073.2597085

[9] Mario Linares-Vásquez, Gabriele Bavota, Carlos Bernal-Cárdenas, Rocco Oliveto, Massimiliano Di Penta, and Denys Poshyvanyk. 2015. Optimizing Energy Consumption of GUIs in Android Apps: A Multi-objective Approach. In *Proceedings of the 10th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*. ACM, 143–154.

[10] Zhe Liu, Chunyang Chen, Junjie Wang, Mengzhuo Chen, Boyu Wu, Xing Che, Dandan Wang, and Qing Wang. 2024. Make LLM a Testing Expert: Bringing Human-like Interaction to Mobile GUI Testing via Functionality-aware Decisions. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE '24)*. ACM, 1222–1234. doi:10.1145/3597503.3639180

[11] Zhe Liu, Cheng Li, Chunyang Chen, Junjie Wang, Boyu Wu, Yawen Wang, Jun Hu, and Qing Wang. 2024. Vision-driven Automated Mobile GUI Testing via Multimodal Large Language Model. arXiv preprint arXiv:2407.03037.

[12] Wellington Oliveira. 2026. An Empirical Analysis of Mobile Energy Consumption Across User Configurations. *arXiv preprint arXiv:2604.25587* (2026).

[13] Wellington Oliveira, Bernardo Moraes, Fernando Castor, and João Paulo Fernandes. 2023. Ebserver: Automating Resource-Usage Data Collection of Android Applications. In *2023 IEEE/ACM 10th International Conference on Mobile Software Engineering and Systems (MOBILESoft)*. 55–59. doi:10.1109/MOBILSoft59058.2023.00014

[14] Rui Pereira, Hugo Matalonga, Marco Couto, Fernando Castor, Bruno Cabral, Pedro Carvalho, Simão Melo de Sousa, and João Paulo Fernandes. 2021. GreenHub: A Large-Scale Collaborative Dataset to Battery Consumption Analysis of Android Devices. *Empirical Software Engineering* 26, 3 (2021), 38. doi:10.1007/s10664-020-09925-5

[15] Gustavo Pinto and Fernando Castor. 2017. Energy Efficiency: A New Concern for Application Software Developers. *Commun. ACM* 60, 12 (2017), 68–75. doi:10.1145/3154384